

Departamento Electrónica

Técnicas Digitales II

Actividad de Formación Practica 1

Tema: Entorno de Desarrollo Integrado STM32CubeIDE,
programación de microcontroladores.

Prof.: Ing. Rubén Darío Mansilla

ATTP: Ing. Lucas Abdala

vencimiento: 30 de mayo de 2025

Grupo 3:

Seco Julian Agustin 51635

1. Elabore un tutorial en soporte digital, que explique cómo se aborda el uso del STM32CubeIDE, que contenga lo que se indica a continuación:

- Instalación del software

STM32CubeIDE

ACTIVE

Save to my ST

Integrated Development Environment for STM32

Get Software

Download databrief

Overview

Documentation

Tools & Software

Product overview

Key Benefits

Description

All features

Get Software

Featured Products

Featured Videos

Recommended for you

Description

STM32CubeIDE is an all-in-one multi-OS development tool, which is part of the STM32Cube software ecosystem.



STM32CubeIDE is an advanced C/C++ development platform with peripheral configuration, code generation, code compilation, and debug features for STM32 microcontrollers and microprocessors. It is based on the Eclipse®/CDT™ framework and GCC toolchain for the development, and GDB for the debugging. It allows the integration of the hundreds of existing plugins that complete the features of the Eclipse® IDE.

STM32CubeIDE integrates STM32 configuration and project creation functionalities from STM32CubeMX to offer all-in-one tool experience and save installation and development time. After the selection of an empty STM32 MCU or MPU, or preconfigured microcontroller or

microprocessor from the selection of a board or the selection of an example, the project is created and initialization code generated. At any time during the development, the user can return to the initialization and configuration of the peripherals or middleware and regenerate the initialization code with no impact on the user code.

STM32CubeIDE includes build and stack analyzers that provide the user with useful information about project status and memory requirements.

STM32CubeIDE also includes standard and advanced debugging features including views of CPU core registers, memories, and peripheral registers, as well as live variable watch, Serial Wire Viewer interface, or fault analyzer.

All features

- Integration of services from STM32CubeMX: STM32 microcontroller, microprocessor, development platform and example project selection, Pinout, clock, peripheral, and middleware configuration, Project creation and generation of the initialization code, Software and middleware completed with enhanced STM32Cube Expansion Packages
- Based on Eclipse®/CDT™, with support for Eclipse® add-ons, GNU C/C++ for Arm® toolchain and GDB debugger

[Read more](#)

Get Software

Part Number	General Description	Supplier	Download	All versions
STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	ST	Get latest	Select version
STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	ST	Get latest	Select version
STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	ST	Get latest	Select version
STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	ST	Get latest	Select version
STM32CubeIDE-Win	STM32CubeIDE Windows Installer	ST	Get latest	Select version

STMicroelectronics recommends always keeping your software up to date



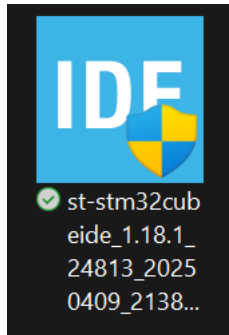
Looking for previous STM32CubeIDE versions?

Please open a case to our support team. We're here to assist you!

[Open a case](#)

Desde el sitio web que indicó el profesor en el aula virtual, luego de realizar el registro, procedemos a elegir la versión adecuada a nuestro ordenador y descargar.

Una vez descargado el archivo zip, lo extraemos y obtenemos el instalador del software

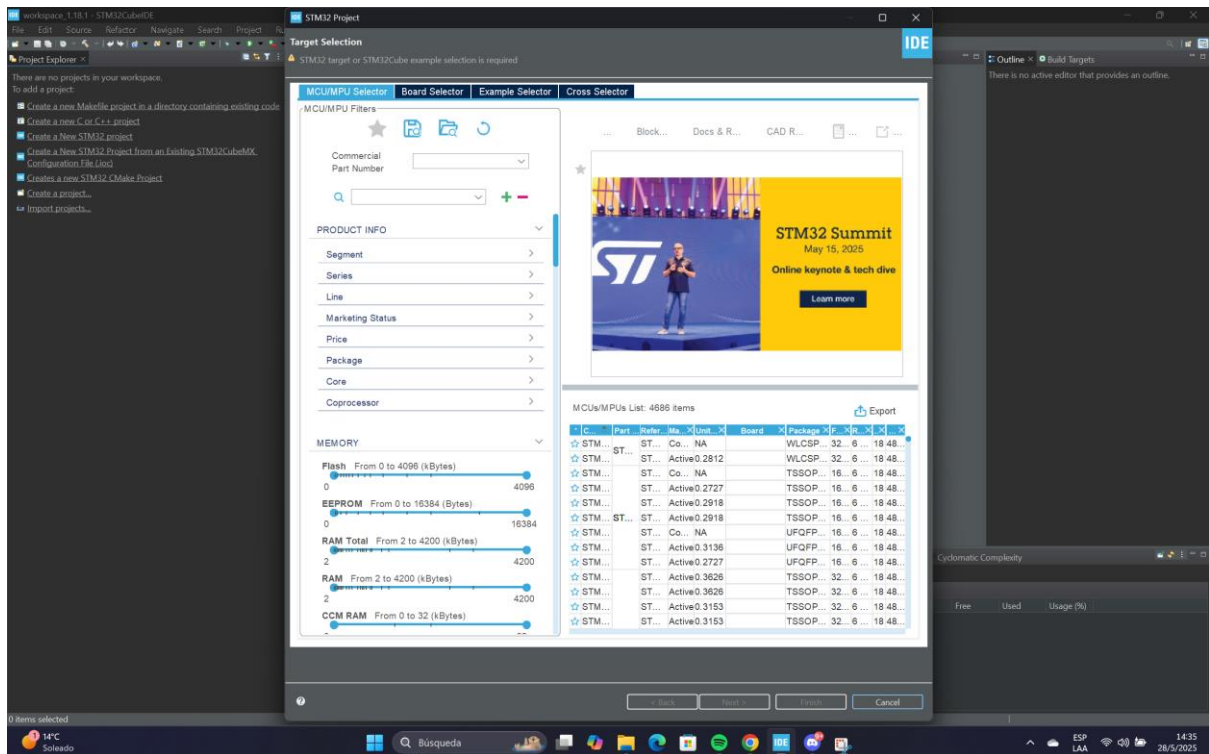


Al que procedemos a ejecutar e instalar.

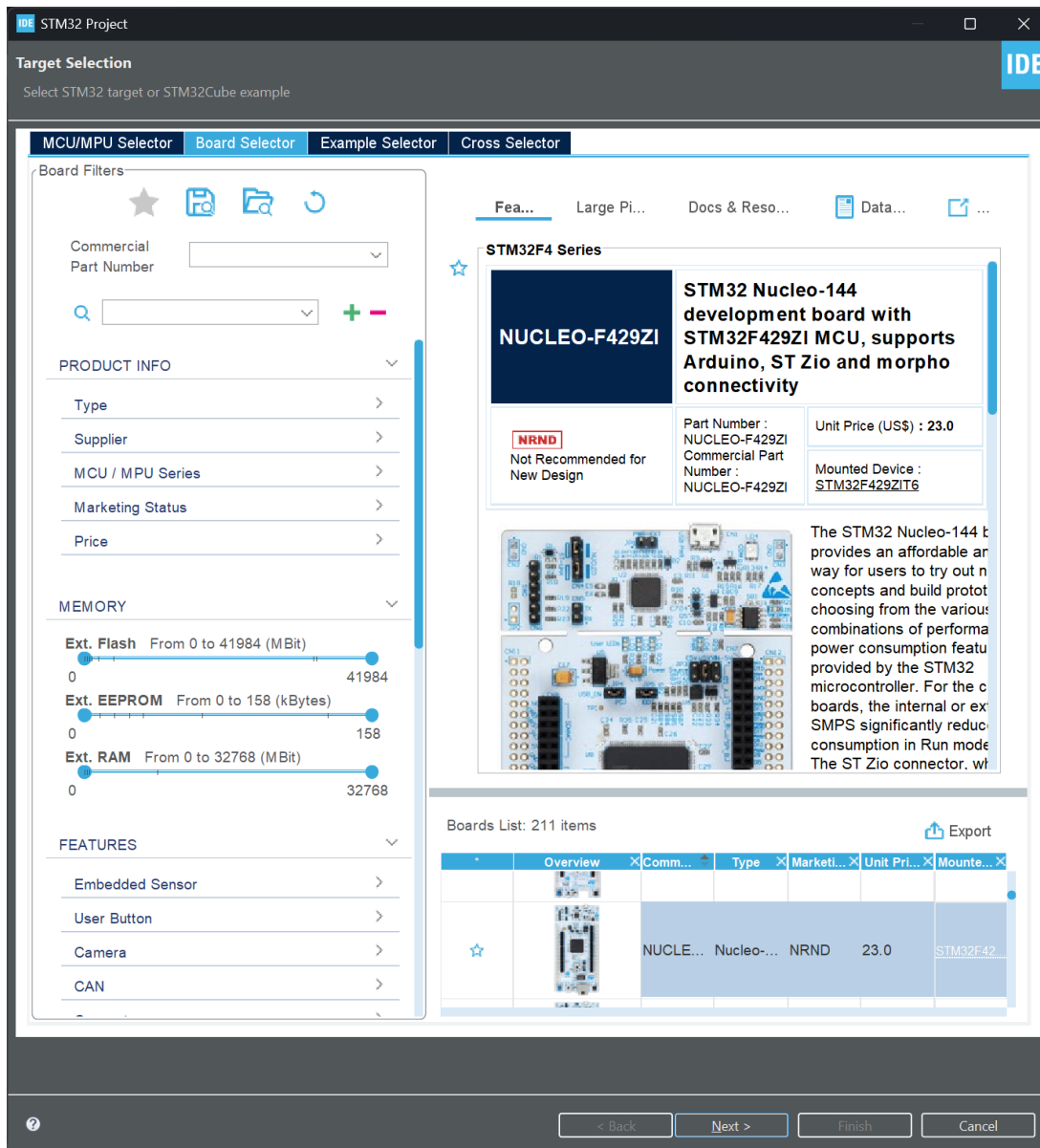
- Creación de un proyecto nuevo desde cero que utilice la plantilla general que ofrece este IDE. Indique las configuraciones básicas para el uso del módulo GPIO correspondientes a los leds y el pulsador de usuario integrados en la placa de desarrollo que utilizaremos en este curso.

Para crear un proyecto nuevo vamos a necesitar iniciar el programa STM32CubeIDE

Una vez ya en el programa, vamos a File > New y seleccionamos "Create a New STM32 project"



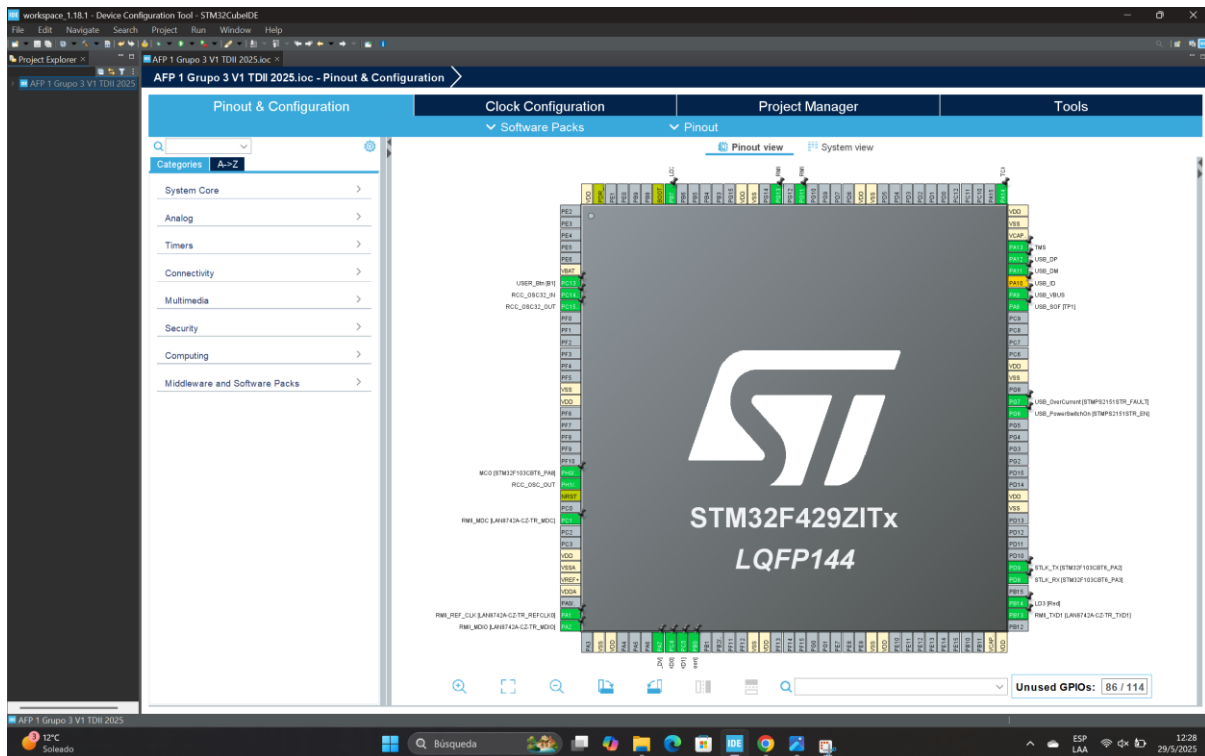
En el siguiente paso, iremos a la pestaña Board Selector, y buscaremos por nombre de placa en nuestro caso NUCLEO-F429ZI



Hacemos clic en “next” y nos llevara a la siguiente ventana en donde vamos a elegir el nombre de nuestro proyecto “AFP 1 Grupo 3 TDII 2025”

Para la configuración del módulo GPIO para led y botón

Se abre automáticamente al iniciar el proyecto el archivo .ioc



Para indicar las configuraciones básicas del modulo GPIO, primero vamos a identificar los pines, ya que las configuraciones de este módulo serían los ajustes mínimos que necesitamos hacer en el software para que los pines funcionen correctamente como entrada o salida. Estas configuraciones las vamos a hacer en la vista grafica del archivo .ioc

Pines:

LED 1 (Green) → PB0

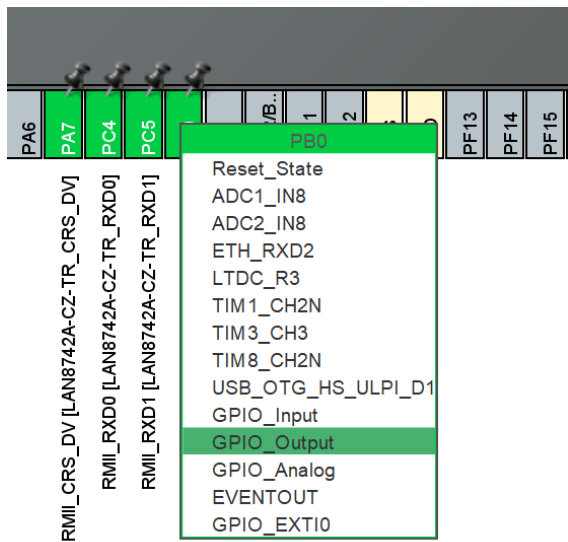
LED 2 (Blue) → PB7

LED 3 (Red) → PB14

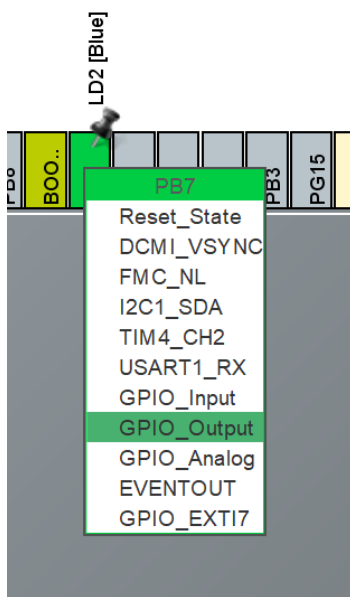
BOTON DE USUARIO (B1) → PC13

Seleccionamos los Leds, pin PB0, PB7, PB14 y configuramos como GPIO_Output.

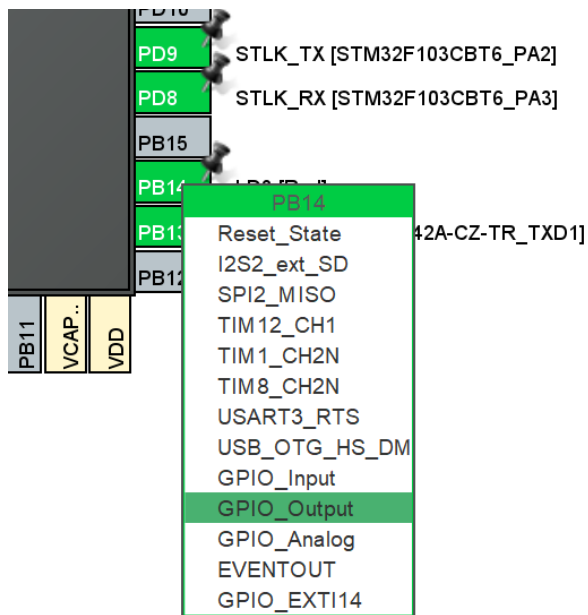
PB0



PB7



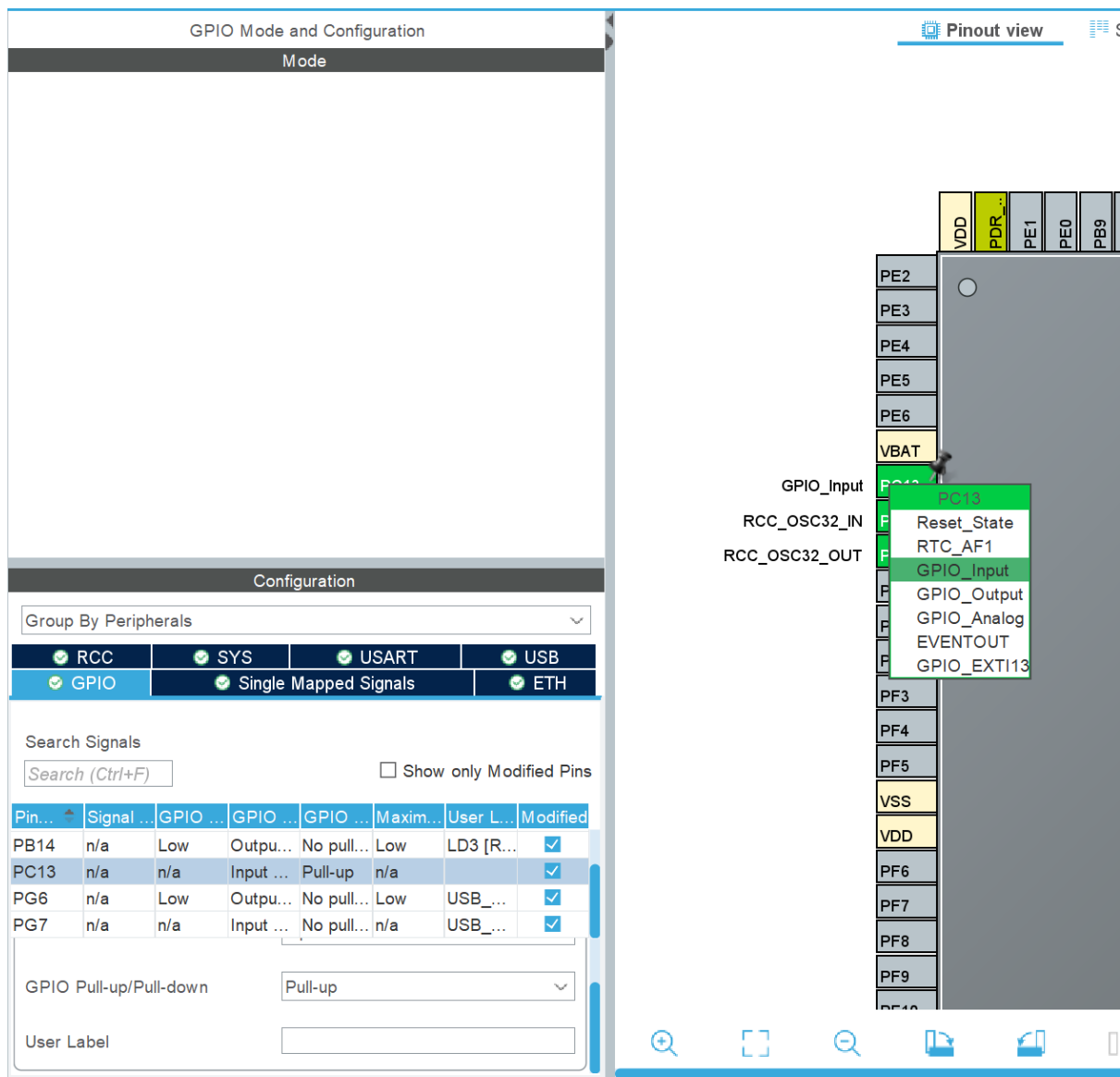
PB14



En el caso del botón de usuario, vamos a seleccionar el PC13 adonde está integrado el botón de usuario de esta placa y vamos a darle la configuración GPIO_Input ya que un botón es un dispositivo de entrada, su función es permitir que el microcontrolador detecte si está siendo presionado. Al configurar el pin de esta manera, se puede leer su estado lógico.

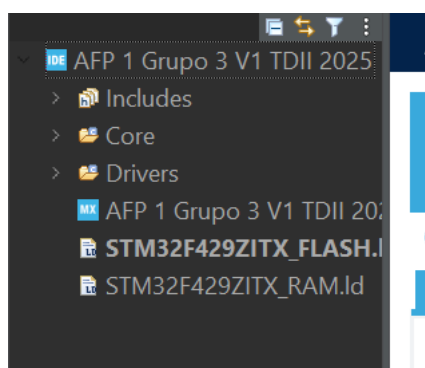
En la configuración de PC13 activaremos el pull-up para mantener el pin en estado lógico alto, cuando el botón no está presionado.

En la siguiente imagen se muestra la designacion del pin y la activacion del pull-up



- Descripción de la estructura del árbol de archivos de un proyecto en STM32CubeIDE y que contiene cada carpeta de esta.

Al generar el proyecto, STM organiza automáticamente los archivos en una estructura jerárquica. De la siguiente manera:



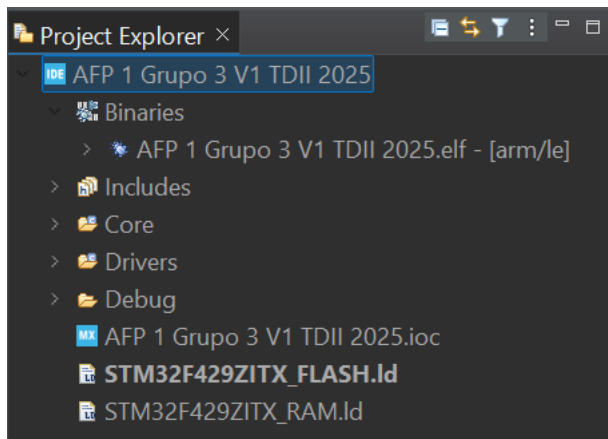
- ♦ Includes: Contiene accesos directos a los archivos de cabecera incluidos en el proyecto
- ♦ Core: Contiene el corazón del proyecto:
 - SRC: Son los archivos fuente como main.c, donde se escribe el código principal del programa
 - INC: Archivos de cabecera (headers) como main.h, donde se declaran funciones, variables globales, etc.
- ♦ Drivers:
 - STM32F4xx_HAL_Driver: Contiene los controladores HAL (Hardware Abstraction Layer) específicos del microcontrolador.
 - CMSIS: define funciones y estructuras de bajo nivel para el núcleo del microcontrolador (capa estándar ARM)}
- ♦ Archivo .ioc: configuración del proyecto, donde se definen los periféricos, pines y configuraciones iniciales. Permite reabrir la vista gráfica de configuración.

- Desarrollo de un proyecto que utilice uno de los leds integrados en la placa y lo haga parpadear cada 250 ms.

Parpadeo de LED PB0 cada 250ms: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/App_1_1_Grupo_3_2025.zip

- Explicación de cómo compilar el proyecto, cuál es el archivo que se genera (código objeto) y donde se ubica en el árbol de archivos de este.

Para compilar el proyecto, abrimos STM32CubeIDE y cargamos el proyecto, clickeamos en “build” de esa manera el IDE compilara todos los archivos fuente como los .c y los .h generando los archivos binarios necesarios para programar la placa.



El compilar se genera un archivo con extension .elf ubicado dentro de la carpeta / debug. Este archivo representa el código objeto que será cargado en la placa NUCLEO-F429ZI

Visualmente también lo podemos ver como “AFP 1 Grupo 3 V1 TDII 2025.elf - [arm/le]”, que se encuentra en la parte superior del árbol de archivos en la sección “Binaries”

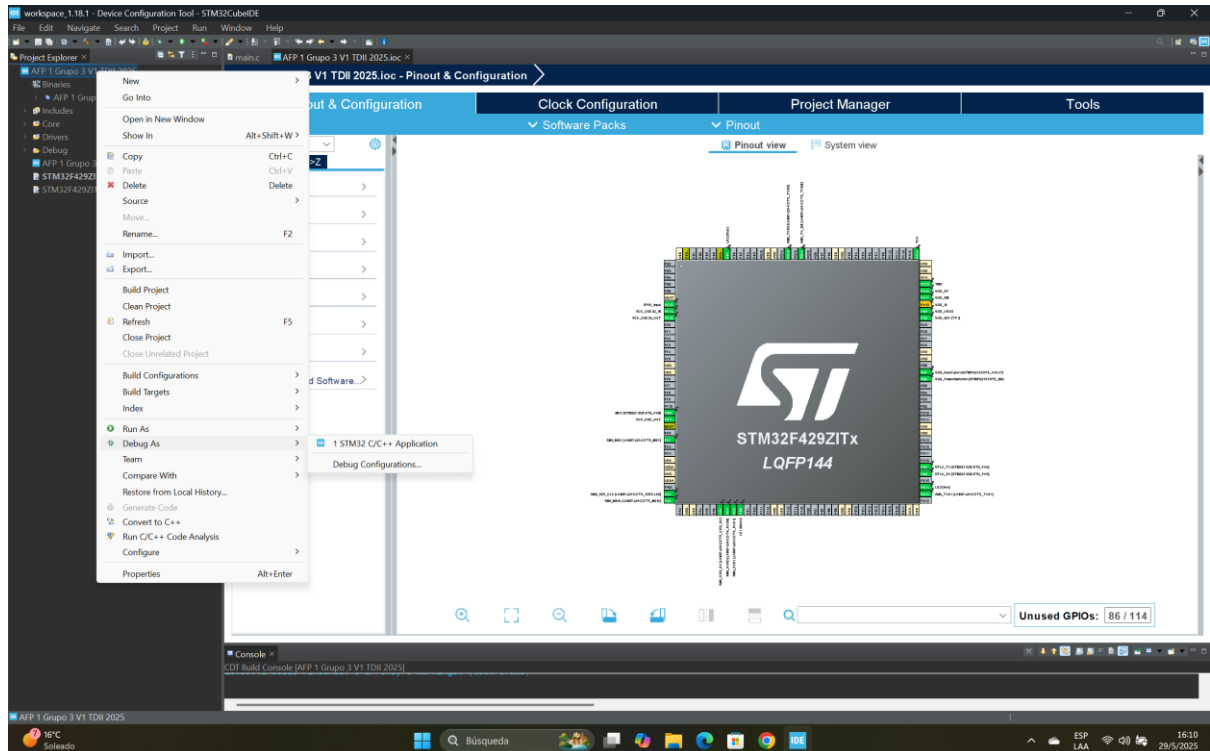
- Explicación sobre cómo configurar el debugger para utilizarlo y cómo se baja el código objeto a la placa target.

Para utilizar el debugger y bajar el código objeto a la placa, se utiliza el programador o debugger integrado “ST-LINK”

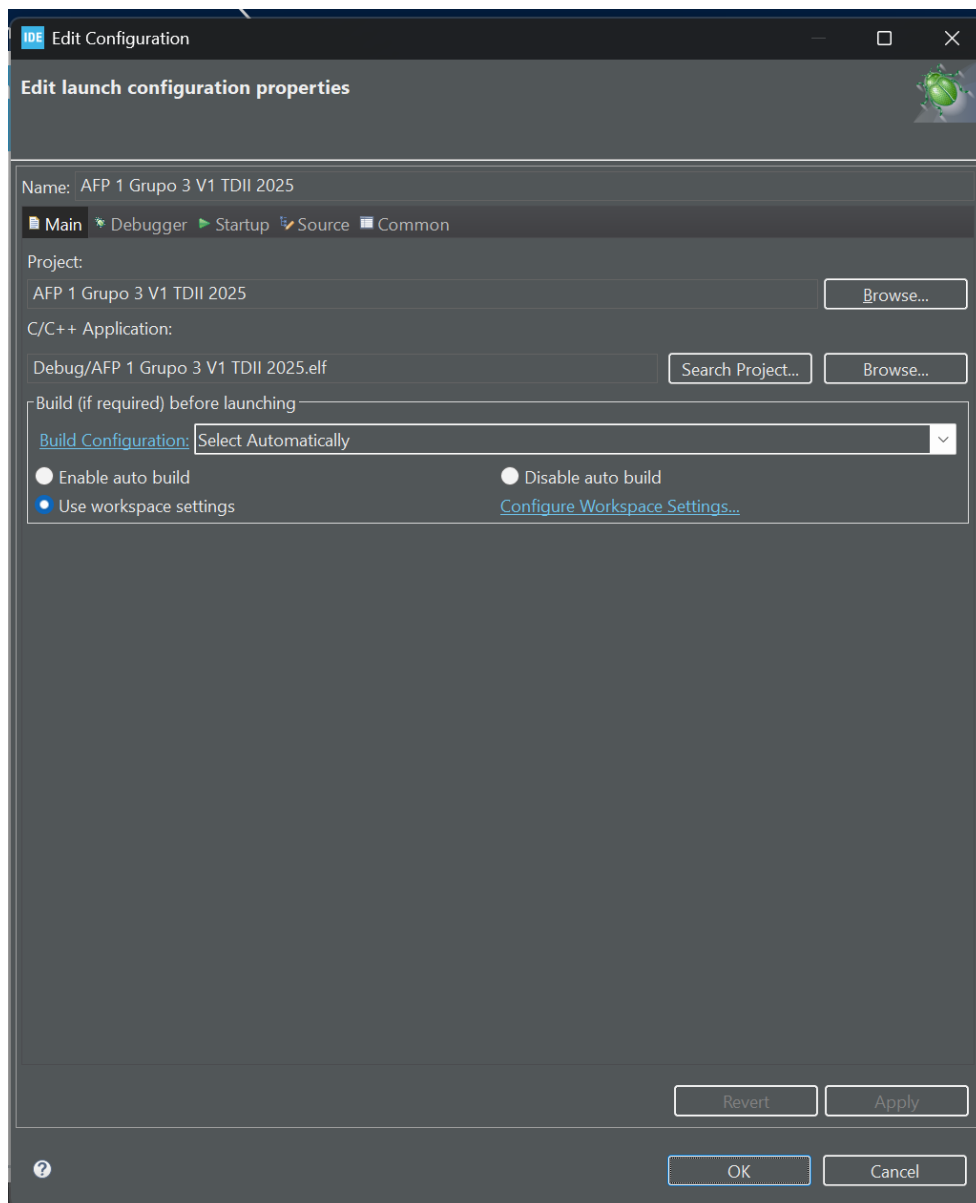
Se conecta la placa a la PC mediante un cable micro USB

En el software

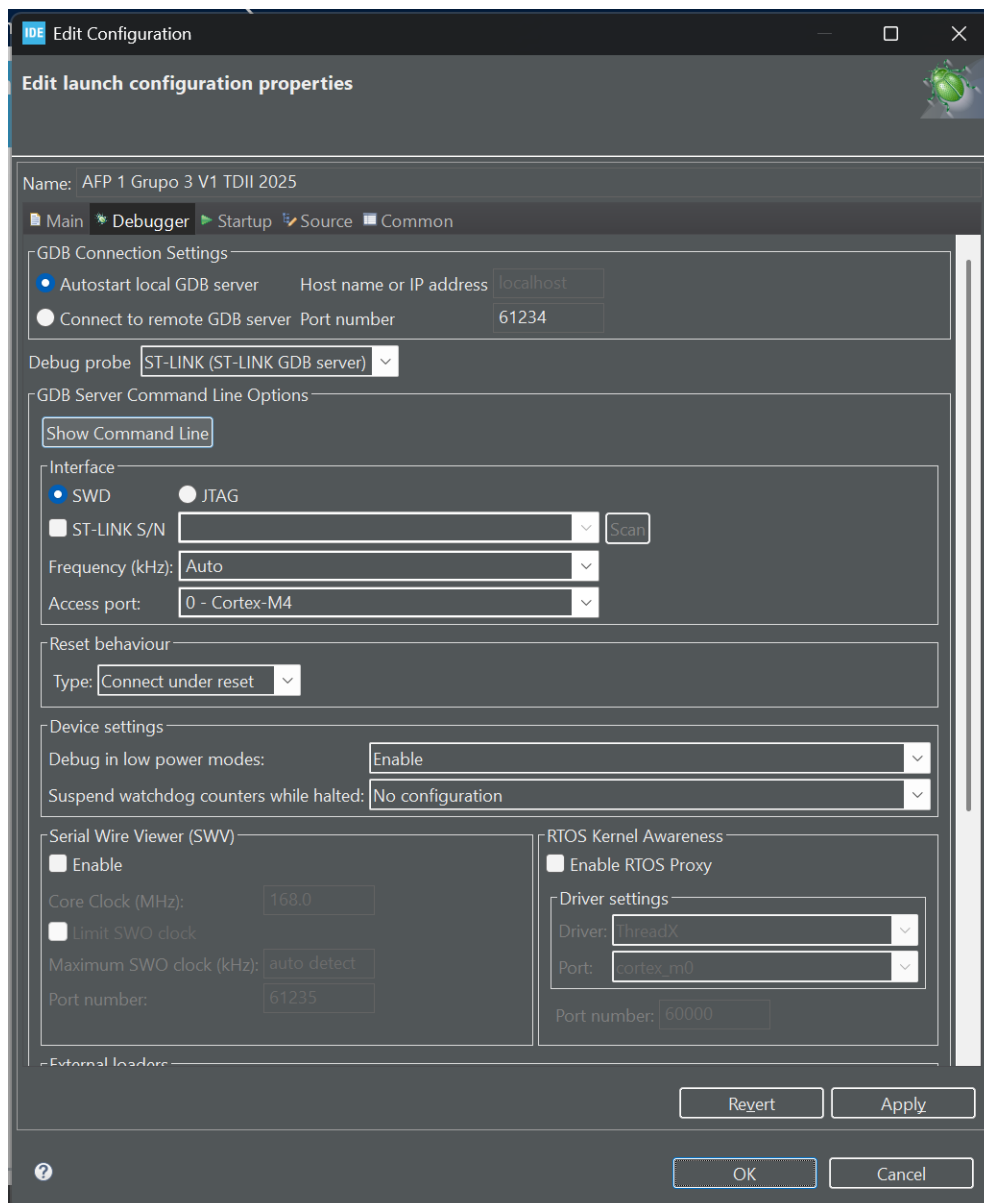
Realizamos click derecho en el proyecto, vamos a “Debug as” y luego a la opción STM32 Cortex-M C/C++ Application.



Nos aparecera la siguiente ventana de configuración



Vamos a la pestaña “Debugger” y nos aparecera lo siguiente



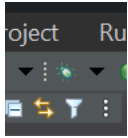
Elegimos la interface SWD, le damos a apply y ok.

Una vez iniciado el proceso de debug:

El archivo .elf es automaticamente cargado en la memoria flash del microcontrolador.

Luego de la carga, el programa se detiene en main () esperando que el usuario inicie la ejecucion

Para comenzar la ejecución pulsamos en el botón “Resume” en la barra de herramientas de debug, el botón verde de la siguiente imagen



Aplicaciones:

El desarrollo de todas las aplicaciones se realizó en el entorno **STM32CubeIDE**, utilizando el lenguaje **C** y haciendo uso del **HAL (Hardware Abstraction Layer)** para el control de los periféricos.

La base del proyecto fue configurada mediante **STM32CubeMX**, donde se habilitaron:

- Los **tres Leds onboard** (usualmente LED1 verde, LED2 azul y LED3 rojo).
- El **pulsador onboard** (usualmente el botón USER).

Para mantener el software escalable y reutilizable, se utilizó un **vector de punteros a pines de LED**. Esto permite que las secuencias se controlen mediante índices y bucles, lo que facilita extender la cantidad de Leds con cambios mínimos.

Las demoras se implementaron con la función `HAL_Delay()` en milisegundos.

Funcionamiento de cada aplicacion:

App 1.1

- ♦ Realiza un **barrido secuencial**: enciende y apaga uno por uno los 3 LEDs, con una alternancia de 200 ms.
- ♦ La lógica es cíclica: LED1 → LED2 → LED3 → LED1...
- ♦ Se usó un **vector de LEDs** y un índice que se va incrementando y reinicia al llegar al final.

Adjunto link de github: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_2_TDII_2025/App_1_1.1_Grupo_3_2025.zip

App 1.2

- ♦ Se parte del mismo comportamiento de App 1.1.

- ♦ Al presionar el pulsador, **invierte la dirección de la secuencia**.
- ♦ Para esto se implementó una **variable de dirección** (+1 o -1) que cambia al detectar una pulsación.
- ♦ Se incorporó una lógica de **detección de flanco** para evitar múltiples detecciones por rebote (con debounce).

Adjunto link de github: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_2_TDII_2025/App_1_1.2_Grupo_3_2025.zip

App 1.3

- ♦ Maneja **cuatro secuencias distintas**, alternando entre ellas con el pulsador.
- ♦ Un contador lleva el número de secuencia actual y se incrementa con cada pulsación.
- ♦ Las secuencias son:
 1. Secuencia de tipo barrido (como App 1.1) a 150 ms.
 2. Todos los LEDs parpadean juntos (300 ms).
 3. Cada LED parpadea con diferente frecuencia (100 / 300 / 600 ms).
 4. LED1 y LED3 parpadean juntos, LED2 parpadea inversamente (150 ms).
- ♦ Se utilizó **una máquina de estados** sencilla con switch-case o if-else, basada en el número de secuencia.

Adjunto link de github: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_2_TDII_2025/App_1_1.3_Grupo_3_2025.zip

App 1.4

- ♦ Hace parpadear los 3 LEDs **simultáneamente**.
- ♦ Cambia la **frecuencia de parpadeo** con el pulsador entre 4 tiempos: 100 / 250 / 500 / 1000 ms.
- ♦ Similar a la App 1.3, se usa un índice para cambiar de tiempo con cada pulsación.
- ♦ Los tiempos están definidos en un **vector de enteros**, y se aplica HAL_Delay(tiempos[indice]) en cada iteración.

Adjunto link de github: https://github.com/Julihubgit/Grupo-3-TDII-2025/blob/main/AFP_2_TDII_2025/App_1_1.4_Grupo_3_2025.zip

